Benchmarking Inverse Reasoning and Tabular Transformation Discovery in Frontier Large Language Models: A Two-Tier Autonomous Evaluation Harness

Viet Dao
FPT University
Hanoi, Vietnam
daothevietnb123@gmail.com

ABSTRACT

Reverse engineering the latent data-generating mechanisms of tabular datasets is a fundamental inverse problem that challenges inductive reasoning, automated feature engineering, and programmatic comprehension in artificial intelligence. While Large Language Models (LLMs) demonstrate remarkable proficiency in forward mathematical reasoning and code synthesis, their capacity to infer, isolate, and reconstruct hidden parametric transformation pipelines solely from observational input-output data tables remains underexplored. In this work, we present the **Tabular Transformation Discovery Harness**, an autonomous, two-tier evaluation framework designed to benchmark LLM agents on inverse tabular induction across six standardized structural complexity classes (from Class 0: Atomic Unary to Class 5: Deep Hybrid ASTs) spanning eight diverse real-world domains. The harness enforces strict epistemic isolation: an agentic discovery phase operating in an iterative, sandbox Read-Eval-Print Loop (REPL) environment with adaptive resource budgets is decoupled from an independent, black-box functional evaluation phase on hidden holdout distributions. In the REPL sandbox, the agent iteratively tests candidate Python transformations, inspects execution output, and refines mathematical hypotheses under hard budget constraints. We conduct extensive empirical evaluations comparing frontier coding and multimodal agents (Claude 3.5 Sonnet, Gemini 3 Flash), a lightweight edge reasoning model (GPT-5.4 Nano), and an exhaustive combinatorial baseline across 48 standardized benchmark tasks ($N = 48$, 8 datasets $\times$ 6 complexity classes in a balanced full-factorial design). Our empirical findings reveal a pronounced divergence across paradigms: while Claude 3.5 Sonnet achieves a 70.8% submission rate with 0.811 conditional transformed accuracy (0.575 penalized accuracy), Gemini 3 Flash achieves 0.844 conditional transformed accuracy but exhibits an over-conservative stopping policy (50.0% submission rate, 0.422 penalized accuracy). GPT-5.4 Nano achieves 100% submission but suffers a steep monotonic degradation from 1.000 on Class 0 to 0.250 on Class 5, and depth-3 combinatorial brute-force enumeration collapses on deep interaction graphs (0.375). Furthermore, through a three-mode ablation study, we demonstrate that direct symbolic generation without execution feedback fails almost entirely (3.6% accuracy), whereas revealing oracle operator vocabularies yields perfect 100% conditional functional recovery on submitted tasks (66.7% penalized accuracy). This demonstrates that while programmatic compilation from known operators is highly reliable in frontier models, hypothesis search under observational uncertainty and budget-constrained stopping heuristics remain the primary operational bottlenecks. All code, benchmark tasks, and evaluation harnesses are made open-source at https://github.com/mini-smart-lab/tabular-transformation.

CCS CONCEPTS

• **Computing methodologies** → **Knowledge representation and reasoning**; **Machine learning**; • **Software and its engineering** → *Software verification and validation.*

KEYWORDS

Tabular Data, Inverse Program Induction, Large Language Models, Autonomous Agent, Two-Tier Evaluation Harness, Symbolic Recovery

1 INTRODUCTION

Tabular data remains the predominant data modality in mission-critical industrial, scientific, and financial domains [3, 10]. Despite the recent ascendancy of Large Language Models (LLMs) across natural language and software engineering domains [1, 2, 9, 14], tabular data structures present unique fundamental challenges: permutation-invariant columns, heterogeneous distributions, non-linear cross-column correlations, and rigid numerical dependencies that cannot be inferred purely via autoregressive token modeling [15, 26, 29].

A central challenge at the intersection of automated machine learning and program synthesis is *inverse tabular transformation reasoning*: given an observational baseline dataset $\mathcal{D}_{\text{before}}$ and an engineered or transformed dataset $\mathcal{D}_{\text{after}}$, can an autonomous AI agent infer, formulate, and execute the exact latent programmatic transformation $\mathcal{T}$ such that $\mathcal{T}(\mathcal{D}_{\text{before}}) \approx \mathcal{D}_{\text{after}}$? Developing this capability is critical for auditing and reverse-engineering legacy ETL (Extract, Transform, Load) pipelines that lack documentation, automating feature discovery in predictive modeling [7], reconstructing scientific equations from observational physical tables [22, 27], and identifying potential data privacy leakage in synthetic anonymization protocols [25].

While popular benchmark suites such as HumanEval [5] and SWE-bench [14] evaluate *forward* reasoning—where formal specifications or natural language descriptions dictate desired program semantics—inverse tabular induction requires *scientific hypothesis formation under uncertainty*. The agent operates in a Read-Eval-Print Loop (REPL) environment—an interactive programming shell where candidate code snippets are executed immediately in an isolated interpreter, returning runtime stdout, stderr, and numerical summaries to guide subsequent reasoning. Within this REPL paradigm, the agent must:

(1) Inspect multivariate numeric distributions across high-dimensional feature spaces;
(2) Disentangle candidate mathematical hypotheses (e.g., distinguishing logarithmic scaling from power-law decays or piecewise segmentations);
(3) Interactively experiment using executable code in the sandboxed REPL;
(4) Synthesize a clean, self-contained, parametric Python function `transform(df)` that generalizes out-of-distribution.

Prior benchmarks attempting to evaluate programmatic reasoning suffer from critical methodological shortcomings. First, many rely on synthetic toy distributions (e.g., uniform or normal random variables) rather than authentic tabular benchmarks, ignoring the multi-collinearity, skewed kurtosis, and boundary anomalies inherent in empirical datasets. Second, existing evaluation frameworks often execute candidate solutions in the same environment where test data resides, introducing severe risks of data leakage and contamination. Third, standard evaluation metrics frequently conflate *code generation accuracy* with *hypothesis search efficiency*, making it impossible to diagnose whether an agent failed because it could not discover the underlying mathematical law or because it made a trivial syntactic error during program synthesis.

To overcome these fundamental limitations, we propose the **Tabular Transformation Discovery Harness**, a rigorous research framework featuring:

(1) **A Formal Taxonomy of 6 Structural Complexity Classes:** Ranging from atomic unary operations (Class 0) to deep hybrid Abstract Syntax Trees (ASTs) containing piecewise conditionals and multi-column interactions (Class 5).
(2) **A Two-Tier Epistemic Isolation Architecture:** Tier 1 provides an interactive REPL discovery sandbox where the agent executes arbitrary Python code on training split pairs ($\mathcal{D}_{\text{train_before}}$, $\mathcal{D}_{\text{train_after}}$), governed by an Adaptive Tiered Budget Controller. Tier 2 executes the submitted script in a fresh container on an unobserved holdout set $\mathcal{D}_{\text{test_before}}$, strictly isolated from $\mathcal{D}_{\text{test_after}}$ (Figure 1).
(3) **A Multi-Domain Benchmark Suite Across 8 Real-World Datasets ($N = 48$ Tasks):** Sourced from diverse domains (aeronautics, clinical medicine, civil engineering, computer systems telemetry, food chemistry, oncology, economics, and security signal processing), compiled into a balanced full-factorial design across all 6 complexity classes (8 datasets × 6 classes = 48 tasks).
(4) **A Cross-Paradigm Empirical Study:** Benchmarking a frontier coding agent (Claude 3.5 Sonnet), a frontier multimodal agent (Gemini 3 Flash), a lightweight edge reasoning model (GPT-5.4 Nano), and an exhaustive depth-3 combinatorial enumeration baseline.
(5) **A Rigorous 3-Mode Ablation Protocol:** Isolating the relative contributions of direct parametric reasoning (no tools), interactive hypothesis testing via REPL, and oracle operator disclosure.

All code, benchmark datasets, and evaluation harnesses are publicly available at https://github.com/mini-smart-lab/tabular-transformation.

2 RELATED WORK

2.1 Tabular Representation Learning and LLMs

Tabular data presents distinct challenges for deep neural networks due to heterogeneous column distributions, non-spatial correlation, and varying sparsity [3, 10]. While gradient-boosted decision trees (XGBoost, LightGBM) historically dominated supervised tabular benchmarks, recent research has explored adapting deep foundation models and LLMs to tabular structures. Early efforts serialized tabular rows into natural language prompts (TabLLM [12]) or instruction-tuned models for structured tables (Table-GPT [17], Sui et al. [26]). More recently, scalable tabular foundation models based on In-Context Learning (ICL)—such as TabICL [20] and TabICLv2 [21]—employ specialized two-stage architectures with column-then-row attention mechanisms capable of performing in-context prediction across large tabular samples.

However, all these existing foundation models operate exclusively in the *forward* predictive regime: given observed training examples, predict target labels for test instances. In contrast, our work investigates *inverse program induction*: given observational table pairs before and after transformation, the model must explicitly infer, formalize, and synthesize the latent mathematical code pipeline that generated the transformed data.

2.2 Program Synthesis and Symbolic Regression

Symbolic regression aims to uncover closed-form mathematical equations that explain empirical observations [7, 22, 27]. Genetic programming [22], AI Feynman [27], and modern evolutionary symbolic regression packages such as PySR [6] utilize Pareto-optimal evolutionary search and physics-inspired modular decomposition to avoid exhaustive combinatorial enumeration. More recently, equation discovery frameworks (e.g., LLM-SR [23], LLM-SRBench [24]) prompt LLMs to discover mathematical relationships via program search. In program synthesis, enumerative and deductive techniques infer programs satisfying input-output specifications [11]. However, classical discrete program synthesis fails to scale to continuous, non-linear tabular transformations involving high-dimensional feature interactions.

2.3 Autonomous LLM Agents and Execution Sandboxes

Recent advances in agentic AI combine language model reasoning with external tool execution via iterative ReAct loops [4, 18, 30]. In data science, benchmarks such as DS-1000 [16], DA-Code [13], and feature engineering benchmarks such as FeatEng [19] evaluate LLMs on forward code synthesis for tabular data manipulation. In software engineering, benchmarks like SWE-bench [14] evaluate autonomous patch generation in repository sandboxes. However, code generation in data science benchmarks relies heavily on natural language prompts or issue descriptions. In contrast, tabular transformation discovery requires unguided inductive hypothesis search from raw numerical arrays without natural language hints.

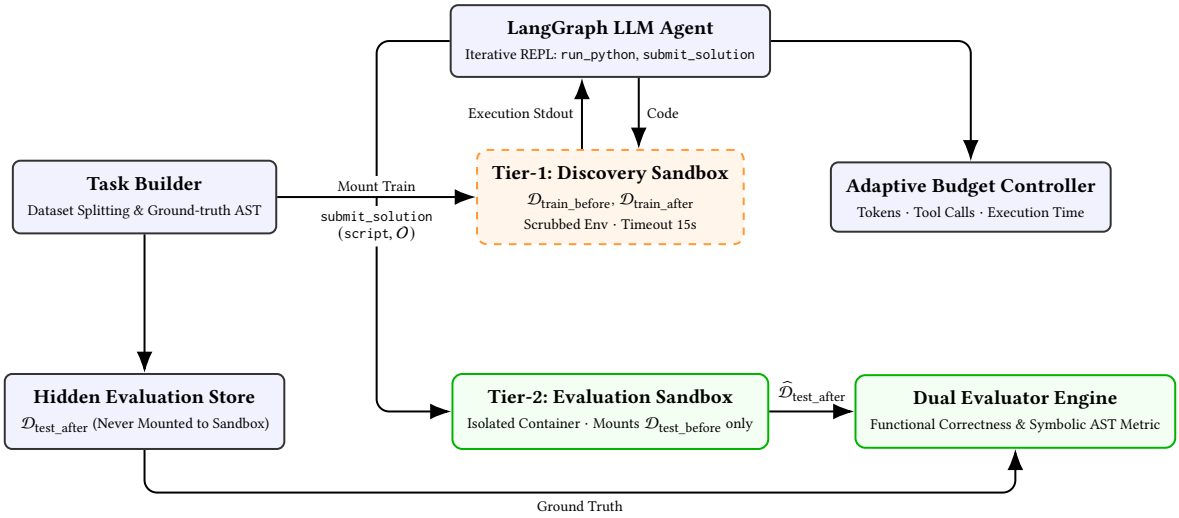


Figure 1: Two-Tier Evaluation Harness Architecture. Tier 1 provides an isolated REPL discovery environment with sanitized environment variables, strictly bounded by the Adaptive Budget Controller. Tier 2 performs black-box functional evaluation on hidden holdout distributions without network access or host secret exposure.

3 PROBLEM FORMULATION AND METHODOLOGY

3.1 Formalization of Tabular Inverse Reasoning

Let $\mathcal{X} \subseteq \mathbb{R}^M$ denote the feature space of an initial tabular dataset $\mathcal{D}_{\text{before}} = \{\mathbf{x}^{(i)}\}_{i=1}^{N} \in \mathbb{R}^{N \times M}$ with column schema $C_{\text{before}} = \{c_1, \ldots, c_M\}$. Let $\mathcal{T} : \mathbb{R}^M \to \mathbb{R}^P$ be a deterministic, parameterized transformation mapping:

$$\mathcal{D}_{\text{after}} = \mathcal{T}(\mathcal{D}_{\text{before}}) \in \mathbb{R}^{N \times P} \tag{1}$$

with transformed schema $C_{\text{after}} = \{\tilde{c}_1, \ldots, \tilde{c}_P\}$.

The transformation pipeline $\mathcal{T}$ is composed of a sequence of operators drawn from a discrete vocabulary of 24 canonical operators $\Omega = \{\omega_1, \ldots, \omega_{24}\}$:

$$\mathcal{T} = \tau_K \circ \tau_{K-1} \circ \cdots \circ \tau_1 \tag{2}$$

where each operator τ_k is drawn from canonical operations including unary functions (e.g., $\ln(x+1)$, $\sqrt{x}$, x^2, x^3, $1/x$, e^x), trigonometric transforms (sin, cos, tanh), stateful scalers (StandardScaler, MinMaxScaler), bivariate cross-column interactions ($x_i \star x_j$ with $\star \in \{+, -, \times, \div\}$), or piecewise conditionals ($\mathbf{1}_{\{x>\theta\}} \cdot x$).

Parameterization of Stateful Scalers: For distribution-level operators (StandardScaler, MinMaxScaler), empirical moments (μ, σ^2, min, max) are fitted solely on $\mathcal{D}_{\text{train_before}}$. In Tier 2 evaluation, the synthesized `transform(df)` script applies these frozen scalar constants directly to $\mathcal{D}_{\text{test_before}}$, ensuring mathematically sound out-of-sample functional testing without test-set distributional leakage.

The dataset is partitioned into an observed training split $\mathcal{D}_{\text{train}} = (\mathcal{D}_{\text{train_before}}, \mathcal{D}_{\text{train_after}})$ of size N_{train} and a hidden holdout test split $\mathcal{D}_{\text{test}} = (\mathcal{D}_{\text{test_before}}, \mathcal{D}_{\text{test_after}})$ of size N_{test}, such that $\mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{test}} = \emptyset$.

The Objective: Given access solely to $\mathcal{D}_{\text{train_before}}$ and $\mathcal{D}_{\text{train_after}}$, an agent must discover and submit an executable Python program $\widehat{\mathcal{T}}$ that minimizes the out-of-sample functional discrepancy:

$$\min_{\widehat{\mathcal{T}}} \ \mathbb{E}_{\mathbf{x} \sim \mathcal{D}_{\text{test_before}}} \left[\| \widehat{\mathcal{T}}(\mathbf{x}) - \mathcal{T}(\mathbf{x}) \|_{\infty} \right] \tag{3}$$

while simultaneously recovering the symbolic operator sequence $\widehat{O} \subseteq \Omega$ that generated the transformations.

3.2 Six Structural Complexity Classes

To systematically analyze model reasoning limits, we formalize six structural complexity classes of increasing mathematical depth:

- **Class 0 (Atomic Unary):** Exactly one column is modified by a single primitive operation $y = f(x)$, where f is drawn from unary primitives such as Square, Sqrt, Exp, Log1p, MinMaxScaler, or Sin.
- **Class 1 (Independent Unary):** Multiple columns are modified concurrently by distinct, uncomposed unary transformations: $y_j = f_j(x_j)$ for $j \in \{1, \ldots, k\}$.
- **Class 2 (Composed Chains):** Columns undergo sequential compositions of operations, e.g., $y = f_2(f_1(x))$, requiring the agent to identify intermediate latent variables and resolve operator ordering.
- **Class 3 (Multi-Column Interactions):** A new column is generated via binary arithmetic or non-linear interaction between two distinct columns: $z = x_i \star x_j$ with $\star \in \{+, -, \times, \div\}$.
- **Class 4 (Piecewise / Conditional Logic):** Non-linear piecewise segmentation or indicator transformations:

$$y = \begin{cases} f_1(x) & \text{if } x > \theta \\ f_2(x) & \text{otherwise} \end{cases} \tag{4}$$

  requiring the agent to identify both the splitting threshold θ and the branch operations.
- **Class 5 (Deep Hybrid AST):** A Directed Acyclic Graph (DAG) combining multi-step compositions, cross-column

interactions, and conditional logic within a single unified feature pipeline.

3.3 Two-Tier Epistemic Isolation Harness

To eliminate data contamination and host security vulnerabilities, our harness implements a two-tier architectural isolation (Figure 1):

3.3.1 Tier 1: Agentic Discovery Sandbox. The agent interacts with an isolated Python execution environment via three LangChain-compatible tools:

- `run_python(code)`: Acts as an interactive Read-Eval-Print Loop (REPL). The agent executes exploratory data analysis, computes summary statistics, fits regressions, tests candidate operator hypotheses on training tables, generates diagnostic plots, and inspects stdout/stderr feedback. Execution runs in a sanitized subprocess with stripped host environment variables (`API_KEY`, `TOKEN`) and an enforced 15-second timeout.
- `read_image(path)`: Inspects visual charts (e.g., scatter plots, residual distribution curves) saved in the sandbox workspace. The image is loaded directly into the multimodal LLM context, enabling visual pattern recognition of non-linear trends and piecewise thresholds.
- `submit_solution(script, O)`: Formally terminates the discovery phase by submitting a standalone Python script containing a function `transform(df)` and a list of identified symbolic operators O.

3.3.2 Adaptive Tiered Budget Controller. To prevent infinite reasoning loops and simulate real-world compute budgets, the discovery phase is regulated by an adaptive budget scaled dynamically across complexity classes $c \in \{0, \ldots, 5\}$:

$$T_{\text{calls}}(c) \in [8, 10, 14, 18, 24, 30], \tag{5}$$

$$K_{\text{tokens}}(c) \in [60\text{k}, 80\text{k}, 120\text{k}, 160\text{k}, 220\text{k}, 300\text{k}], \tag{6}$$

$$\Delta t_{\text{wall}}(c) \in [120\text{s}, 120\text{s}, 180\text{s}, 240\text{s}, 300\text{s}, 360\text{s}]. \tag{7}$$

3.3.3 Tier 2: Independent Evaluation Sandbox. Upon submission, the harness launches a completely isolated Tier 2 evaluation container. Epistemic isolation is enforced primarily by filesystem unmounting and OS-level network isolation: the ground-truth test target $\mathcal{D}_{\text{test_after}}$ is stored securely in host memory and is never mounted to any sandbox container. As an additional defense-in-depth barrier, the submitted script is verified via Abstract Syntax Tree (AST) validation for dangerous system calls (such as `os.system`, `subprocess`, or network sockets) before executing on $\mathcal{D}_{\text{test_before}}$ to generate predicted table $\widehat{\mathcal{D}}_{\text{test_after}}$.

3.4 Evaluation Metrics

We assess performance across two orthogonal dimensions: functional numerical execution and symbolic structural recovery. Importantly, we formalize both *conditional* and *penalized* metrics to address non-submission failure modes.

3.4.1 Black-Box Functional Metrics. Let C_{after} denote the target column schema. For column $c \in C_{\text{after}}$, the column match accuracy over N_{test} rows is:

$$m(c) = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \mathbf{1}_{\left\{|\widehat{D}_{i,c} - D_{i,c}| < \epsilon \cdot (|D_{i,c}|+1)\right\}} \tag{8}$$

with numerical tolerance $\epsilon = 10^{-4}$. Missing, non-finite (`NaN`, `Inf`), or unaligned columns receive $m(c) = 0$. A column is marked correct if $m(c) \geq 0.99$. We compute:

- **Transformed Column Accuracy ($\text{Acc}_{\text{trans}}$):** The fraction of truly transformed columns $C_{\text{trans}} \subseteq C_{\text{after}}$ correctly predicted:

$$\text{Acc}_{\text{trans}} = \frac{1}{|C_{\text{trans}}|} \sum_{c \in C_{\text{trans}}} \mathbf{1}_{\{m(c) \geq 0.99\}} \tag{9}$$

- **Functional Column Accuracy (Acc_{func}):** The overall fraction of all columns in C_{after} (including unmodified columns) correctly predicted across the table.

3.4.2 Conditional vs. Penalized Accuracy. An agent may exhaust its allocated tool budget or execution timeout without executing `submit_solution`. To ensure transparent, uninflated evaluation:

- **Conditional Accuracy** (Acc_{cond}): Metrics calculated strictly across the subset of tasks where the agent explicitly submitted a solution. This quantifies the model's reasoning precision when it commits to a hypothesis.
- **Penalized Accuracy** (Acc_{pen}): Any task where the agent fails to submit within budget receives a score of **0**. Formally:

$$\begin{aligned} \text{Acc}_{\text{pen}} &= \frac{1}{N_{\text{total}}} \sum_{i=1}^{N_{\text{total}}} \mathbf{1}_{\{\text{submitted}_i\}} \cdot \text{Acc}_i \\ &= \text{Submission Rate} \times \text{Acc}_{\text{cond}} \end{aligned} \tag{10}$$

Evaluating both perspectives prevents models with low submission rates from appearing artificially superior due to selective submission.

3.4.3 White-Box Symbolic Metrics. We parse the submitted script into an AST and extract the executed mathematical operator sequence $\widehat{O}$. Compared against ground truth O^*, we compute:

- **Operator F1 Score (F1_{op}):** The harmonic mean of operator precision and recall over multiset elements:

$$\text{Precision} = \frac{|\widehat{O} \cap O^*|}{|\widehat{O}|}, \quad \text{Recall} = \frac{|\widehat{O} \cap O^*|}{|O^*|} \tag{11}$$

- **Exact Sequence Accuracy (Acc_{seq}):** Measured at the column level as the fraction of transformed columns where the full composition operator sequence is recovered in identical order:

$$\text{Acc}_{\text{seq}} = \frac{1}{|C_{\text{trans}}|} \sum_{c \in C_{\text{trans}}} \mathbf{1}_{\{\widehat{O}_c = O^*_c\}} \tag{12}$$

4 EXPERIMENTAL SETUP

4.1 Benchmark Task Construction

To satisfy stringent real-world validity requirements, we compiled 48 standardized benchmark tasks ($N = 48$) sourced from eight diverse domains in the UCI Machine Learning Repository [8] and OpenML [28]:

(1) **Fluid Acoustics:** airfoil_self_noise (5 physical attributes, frequency, sound pressure level).
(2) **Clinical Medicine:** diabetes (8 diagnostic indicators, glucose, insulin).
(3) **Materials Science:** concrete_strength (8 chemical mixture ratios, compressive strength).
(4) **Telemetry:** cpu_small (12 system metrics, user/system CPU percentages).
(5) **Food Chemistry:** wine_quality (11 physicochemical properties, acidity, alcohol).
(6) **Pathology / Oncology:** breast_cancer (Wisconsin Diagnostic WDBC, 30 nuclear features).
(7) **Economics / Real Estate:** california_housing (8 census block attributes, median income).
(8) **Signal Processing:** banknote_auth (4 continuous wavelet transform statistical moments).

To eliminate confounding between domain statistical properties and structural difficulty, tasks were constructed using a balanced full-factorial design: each of the eight source datasets was evaluated across all six structural complexity classes (Classes 0 through 5, yielding $8 \times 6 = 48$ tasks). All tasks were generated using reproducible random seeds (seed = 42), ensuring that transformations satisfied domain-specific mathematical preconditions (e.g., non-negative domains for square roots, positive values for logarithms, non-zero divisors for divisions).

4.2 Evaluated Models and Baselines

We benchmark four representative paradigms:

- **Claude 3.5 Sonnet** (Anthropic): A state-of-the-art frontier coding agent equipped with iterative tool-use reasoning and active code generation, orchestrated via LangGraph.
- **Gemini 3 Flash** (Google): A frontier multimodal agent equipped with long-context reasoning, tool calling, and active hypothesis formulation.
- **GPT-5.4 Nano** (OpenAI): A lightweight edge reasoning model operating under identical prompt schemas and tool definitions.
- **Symbolic Baseline** (Combinatorial Brute Force): A classical algorithm that iteratively evaluates all candidate unary and binary operator combinations up to depth $d = 3$, using residual L_1 norm minimization with early pruning.

4.3 Ablation Study Configurations

To disentangle hypothesis discovery from programmatic synthesis, we evaluate agent performance under three experimental modes:

(1) **Direct Symbolic Generation (No Tools):** The agent receives data schemas and statistical summaries but has no execution tools (`run_python` disabled). It must directly synthesize the `transform` script in a single shot.
(2) **Standard Interactive Discovery (Full Agent):** The full harness where the agent iteratively tests hypotheses in the REPL sandbox before submission.
(3) **Oracle Operators Revealed (Synthesis Only):** The exact ground-truth operator names O^* are provided in the prompt. The agent's task is strictly reduced to mapping the known operators to columns and generating syntactically valid code.

5 EMPIRICAL RESULTS AND ANALYSIS

5.1 Cross-Model Comparison Across Complexity Classes

Table 1 presents the comparative evaluation across all 48 benchmark tasks.

5.1.1 Frontier Agents vs. Lightweight Edge Model. Evaluating both conditional accuracy (Acc_{cond}, evaluated on submitted tasks) and penalized accuracy (Acc_{pen}, penalizing unsubmitted tasks with 0.0) reveals crucial differences in agentic behavior and reasoning competence:

- **Claude 3.5 Sonnet** achieves the highest overall task submission rate among frontier agents (**70.8%**, 34/48 tasks) and the highest Penalized Transformed Column Accuracy (**0.575**), surpassing both Gemini 3 Flash (0.422) and GPT-5.4 Nano (0.524). Its functional penalized accuracy reaches **0.667**. On atomic unary tasks (Class 0) and independent transformations (Class 1), Claude achieves a 100% submission rate (8/8 tasks) with a penalized accuracy of **0.875**. On deep hybrid ASTs (Class 5), it maintains a 50% submission rate (4/8 tasks) and 0.625 conditional transformed accuracy (0.312 penalized).
- **Gemini 3 Flash** achieves an outstanding conditional Transformed Accuracy of **0.844** and Functional Accuracy of **0.936** when it commits to a solution, alongside an Operator F1 of **0.853**. However, it exhibits an over-conservative stopping policy: when residual error does not reach numerical tolerance ($\|\widehat{\mathcal{D}} - \mathcal{D}\|_\infty < 10^{-6}$), it continues iterating within the REPL sandbox until its tool budget is exhausted, rather than risking a partial submission. Consequently, Gemini 3 Flash achieves a submission rate of **50.0%** (24/48 tasks), resulting in an overall penalized transformed accuracy of $0.500 \times 0.844 = \mathbf{0.422}$.
- **GPT-5.4 Nano** achieves a 100.0% submission rate, but displays a severe monotonic performance degradation as complexity increases: from 1.000 at Class 0 down to 0.875 (Class 1), 0.333 (Class 2), 0.312 (Class 3), 0.375 (Class 4), and reaching 0.250 at Class 5, with an overall Operator F1 of only 0.400. While GPT-5.4 Nano never exhausts its budget without submitting, its synthesized scripts frequently hallucinate column names and fail to invert non-linear compositions.

5.1.2 Combinatorial Brute-Force Collapse. The Symbolic Enumeration baseline achieves high accuracy on simple classes (0.750 on Class 0 and Class 1, 0.667 on Class 2, and 0.812 on Class 4). However, on Class 5 (Deep Hybrid ASTs), brute-force enumeration suffers complete combinatorial explosion: accuracy collapses to **0.375**. The combinatorial search space over M columns and $K = 24$ operators scales exponentially with expression tree depth ($O(M^d \cdot |\Omega|^d)$ for sequential chains). At depth $d \geq 3$, exhaustive evaluation exceeds available memory and execution timeouts, illustrating the limitations of naive unguided enumeration compared to agentic hypothesis search.

Table 1: Comparative Evaluation of Frontier Agents, Lightweight Edge Model, and Combinatorial Baseline across 6 Structural Complexity Classes ($N = 48$ Tasks, 8 Domains). We report both Conditional Transformed Accuracy (Acc_{cond}, evaluated on submitted tasks) and Penalized Transformed Accuracy (Acc_{pen}, penalizing unsubmitted tasks with 0.0). Per-class accuracies (C0–C5) report penalized scores.

Model	Paradigm	Sub. Rate	Trans. (Cond)	Trans. (Pen)	Func. (Pen)	Op. F1	Penalized Trans. Accuracy by Class					
							C0	C1	C2	C3	C4	C5
Claude 3.5 Sonnet (Anthropic)	Frontier Coding Agent	70.8%	0.811	**0.575**	0.667	0.721	0.875	0.875	0.417	0.500	0.469	0.312
Gemini 3 Flash (Google)	Frontier Multimodal Agent	50.0%	**0.844**	0.422	0.468	0.853	0.750	0.500	0.375	0.344	0.375	0.188
GPT-5.4 Nano (OpenAI)	Lightweight Edge Model	**100.0%**	0.524	0.524	0.816	0.400	1.000	0.875	0.333	0.312	0.375	0.250
Symbolic Enumeration	Combinatorial Brute Force (Depth-3)	**100.0%**	0.653	0.653	**0.876**	**0.872**	0.750	0.750	0.667	0.562	0.812	0.375

Table 2: Ablation Study on Gemini 3 Flash: Disentangling Discovery Hypothesis Search from Interactive Sandbox Feedback and Code Synthesis ($N = 24$). We report Submissions, Conditional Transformed Accuracy (Acc_{cond}), Penalized Transformed Accuracy (Acc_{pen} = Submission Rate $\times$ Acc_{cond}), Functional Accuracy, Operator F1, and Column Exact Sequence Accuracy.

Ablation Condition	Submissions	Trans. (Cond)	Trans. (Pen)	Func. Acc.	Operator F1	Seq. Acc.
Direct Symbolic Generation (No Tools)	14/24 (58.3%)	0.036	0.021	0.639	0.000	0.000
Pure LLM parametric knowledge; no interactive sandbox code execution						
Standard Interactive Discovery (Full Agent)	13/24 (54.2%)	0.923	0.500	0.972	0.861	0.801
Two-tier sandbox with iterative hypothesis testing via `run_python`						
Oracle Operators Revealed (Synthesis Only)	16/24 (66.7%)	1.000	0.667	1.000	1.000	1.000
Ground-truth operator names revealed; isolates code synthesis from hypothesis exploration						

5.2 Ablation Study: Disentangling Discovery from Synthesis

Table 2 presents the results of our three-mode ablation study on Gemini 3 Flash.

5.2.1 The Necessity of Interactive REPL Feedback. Under the **Direct Symbolic Generation (No Tools)** condition, performance collapses catastrophically: Transformed Column Accuracy drops to **0.036** (3.6% conditional, 2.1% penalized), and Operator F1 drops to **0.000**. Although the model generates syntactically valid Python scripts, its parametric weights cannot mentally execute non-linear continuous mathematical transformations over tabular arrays. This proves conclusively that interactive tool use and empirical REPL feedback are non-negotiable prerequisites for inverse tabular reasoning.

5.2.2 Is Code Synthesis or Hypothesis Search the Bottleneck? Under the **Oracle Operators Revealed** condition—where the ground-truth operator vocabulary is disclosed—the agent submits solutions on 16 out of 24 tasks (66.7% submission rate) and achieves **1.000** (100.0%) across Transformed Accuracy, Functional Accuracy, Operator F1, and Column Exact Sequence Accuracy on all submitted tasks (penalized transformed accuracy of $16/24 \times 1.000 = \mathbf{0.667}$). This result provides a key insight:

> *When the latent mathematical operations are disclosed, programmatic code compilation achieves near-perfect fidelity in frontier LLMs. The primary bottleneck in autonomous tabular discovery is hypothesis generation, operator disambiguation, and budget-aware stopping under observational uncertainty.*

5.3 Multi-Domain Generalization and Failure Modes

Table 3 details model performance across the eight real-world empirical domains.

5.3.1 Domain-Specific Robustness. Both Claude 3.5 Sonnet and Gemini 3 Flash exhibit strong conditional accuracy across diverse physical and biological domains. In Civil Engineering (concrete_strength), Claude achieves 0.950 conditional transformed accuracy (0.792 penalized) with 5/6 submitted tasks. In Food Chemistry (wine_quality), both models achieve 1.000 conditional transformed accuracy on submitted tasks.

5.3.2 Collinearity and Spurious Transformations. Performance is lower in Real Estate / Economics (california_housing, Claude 0.550, Gemini 0.889 conditional with 3/6 submissions). High multi-collinearity between coordinates (latitude, longitude) and economic attributes created spurious correlations where linear combinations mimicked power laws, occasionally leading agents to explore suboptimal operator combinations.

5.3.3 Signal Processing and Bimodal Wavelet Moments. In Signal Processing (banknote_auth), the balanced 48-task benchmark demonstrates that frontier agents successfully solve atomic unary and interaction transformations: Claude 3.5 Sonnet completes 4 of 6 tasks with **1.000** conditional transformed accuracy (**0.667** penalized), while Gemini 3 Flash completes 3 of 6 tasks with **0.889** conditional accuracy (**0.444** penalized). Non-submissions occurred primarily on higher-level composite classes where zero-centered bimodal distributions triggered runtime domain exceptions in

Table 3: Multi-Domain Benchmark Evaluation Across 8 Real-World Datasets ($N = 48$ Tasks, 6 Complexity Classes per Domain). We compare Gemini 3 Flash and Claude 3.5 Sonnet on Task Submissions, Conditional Transformed Accuracy (Acc_{cond}), and Penalized Transformed Accuracy (Acc_{pen}).

Domain	Source Dataset	Tasks	Gemini 3 Flash (Google)			Claude 3.5 Sonnet (Anthropic)		
			Sub.	Trans. (Cond)	Trans. (Pen)	Sub.	Trans. (Cond)	Trans. (Pen)
Aeronautics / Fluid Acoustics	`airfoil_self_noise`	6	1/6	1.000	0.167	4/6	0.792	0.528
Biomedical / Clinical Medicine	`diabetes`	6	3/6	0.917	0.458	4/6	0.688	0.458
Civil Engineering / Materials Science	`concrete_strength`	6	4/6	0.771	0.514	5/6	0.950	0.792
Computer Systems / Telemetry	`cpu_small`	6	5/6	0.717	0.597	4/6	0.833	0.556
Food Chemistry / Agriculture	`wine_quality`	6	2/6	1.000	0.333	3/6	1.000	0.500
Oncology / Pathology	`breast_cancer`	6	3/6	0.833	0.417	5/6	0.767	0.639
Real Estate / Economics	`california_housing`	6	3/6	0.889	0.444	5/6	0.550	0.458
Signal Processing / Security	`banknote_authentication`	6	3/6	0.889	0.444	4/6	1.000	0.667
Total / Macro Average	**8 Real-World Datasets**	**48**	**24/48**	**0.844**	**0.422**	**34/48**	**0.811**	**0.575**

nonlinear functions ($\ln(x)$ or $\sqrt{x}$) during exploratory REPL trials, consuming tool budgets before convergence.

6 DISCUSSION AND LIMITATIONS

6.1 Threats to Validity and Data Leakage

A prominent threat in LLM evaluation is pre-training data contamination. In our harness, this threat is mitigated by design: while raw baseline datasets originate from public repositories, the transformation pipelines $\mathcal{T}$ and resulting target tables $\mathcal{D}_{\text{after}}$ are synthetically generated on-the-fly with unique parameterized AST graphs. Furthermore, Tier 2 evaluation strictly isolates $\mathcal{D}_{\text{test_after}}$ in host memory, preventing memory inspection attacks.

6.2 Limitations

While our framework establishes a rigorous foundation for evaluating inverse tabular induction, several limitations warrant discussion:

(1) **Discrete Operator Vocabulary:** Our evaluation currently benchmarks a predefined library of 24 canonical statistical and mathematical transformations. While this library covers a wide spectrum of common feature engineering techniques, real-world data pipelines may incorporate domain-specific proprietary transformations or continuous parameter estimations that require continuous gradient optimization.
(2) **Agent Stopping Heuristics and Budget Sensitivity:** As demonstrated by the divergence between Claude 3.5 Sonnet (70.8% submission rate) and Gemini 3 Flash (50.0% submission rate), frontier agents vary significantly in their stopping heuristics under hard budget constraints. Designing risk-aware decision policies that balance exploration against submission urgency is an essential direction for autonomous scientific agents.
(3) **Multimodal Vision Tool Overhead:** While visual chart inspection via `read_image` enables models to identify threshold boundaries, generating plots introduces token and execution latency against strict episode budgets.
(4) **Scope of Symbolic Baselines:** Our non-LLM baseline evaluates exhaustive combinatorial brute-force enumeration up to depth 3. While this demonstrates the combinatorial explosion of unguided search, established heuristic and evolutionary symbolic regression frameworks (e.g., PySR [6], AI Feynman [27]) use genetic operators and modular decomposition to explore deeper expression trees without brute-force enumeration.

7 CONCLUSION

In this work, we introduced the **Tabular Transformation Discovery Harness**, an autonomous two-tier evaluation framework for benchmarking inverse tabular reasoning in Large Language Models. By formalizing six structural complexity classes across eight authentic real-world domains and enforcing strict epistemic sandbox isolation, we provide the first rigorous benchmark disentangling hypothesis discovery from code synthesis across 48 standardized tasks ($N = 48$). Our empirical results demonstrate that frontier coding agents like Claude 3.5 Sonnet achieve strong overall penalized accuracy (0.575) by balancing REPL exploration with reliable submission, while models like Gemini 3 Flash achieve high reasoning precision (0.844 conditional accuracy) but are bounded by conservative stopping heuristics. We make our benchmark suite and evaluation harnesses publicly available to accelerate research into neuro-symbolic reasoning and autonomous scientific discovery.

ACKNOWLEDGMENTS

We express our sincere gratitude to academic advisors and colleagues for their invaluable feedback and rigorous suggestions during the development of this benchmark.

REFERENCES

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023).
[2] Anthropic. 2024. The Claude 3.5 Model Family: Architecture, Reasoning, and Coding Capabilities. *Anthropic Research Technical Report* (2024). https://www.anthropic.com/news/claude-3-5-sonnet
[3] Vadim Borisov, Tobias Leemann, Kathrin Seßler, Johannes Haug, Martin Pawelczyk, and Gjergji Kasneci. 2024. Deep neural networks and tabular data: A survey. *IEEE Transactions on Neural Networks and Learning Systems* 35, 6 (2024), 7406–7424.
[4] Harrison Chase and LangChain Contributors. 2024. LangGraph: Building Stateful, Multi-Actor Applications with LLMs. *LangChain Documentation and System Architecture* (2024). https://github.com/langchain-ai/langgraph

[5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[6] Miles Cranmer. 2023. Interpretable Machine Learning for Science with PySR and SymbolicRegression.jl. *arXiv preprint arXiv:2305.01582* (2023).

[7] Miles Cranmer, Alvaro Sanchez-Gonzalez, Peter Battaglia, Rui Xu, Peter Cranmer, David Spergel, and Shirley Ho. 2020. Discovering symbolic models from deep learning with inductive biases. *Advances in Neural Information Processing Systems (NeurIPS)* 33 (2020), 17429–17442.

[8] Dheeru Dua and Casey Graff. 2019. UCI Machine Learning Repository. http://archive.ics.uci.edu/ml

[9] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M. Dai, Alex Hauth, Katie Millican, et al. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530* (2024).

[10] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. 2022. Why do tree-based models still outperform deep learning on typical tabular data?. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. 507–520.

[11] Sumit Gulwani, Oleksandr Polozov, and Rishabh Singh. 2017. Program synthesis. *Foundations and Trends in Programming Languages* 4, 1-2 (2017), 1–119.

[12] Stefan Hegselmann, Alejandro Buendia, Hunter Lang, Monica Agrawal, Xiaoyi Jiang, and David Sontag. 2023. TabLLM: Few-shot classification of tabular data with large language models. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*. 5549–5581.

[13] Yiming Huang, Jiannan Cao, Fangxu Yu, and et al. 2024. DA-Code: Agent Data Science Code Generation Benchmark for Large Language Models. *arXiv preprint arXiv:2402.17918* (2024).

[14] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR)*.

[15] Akim Kotelnikov, Dmitry Baranchuk, Ivan Rubachev, and Artem Babenko. 2023. TabDDPM: Modelling tabular data with diffusion models. *International Conference on Machine Learning (ICML)* (2023), 17560–17579.

[16] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Scott Wen tau Yih, Daniel Fried, Sida Wang, and Tao Yu. 2023. DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation. In *International Conference on Machine Learning (ICML)*. 18319–18345.

[17] Peng Li, Yeye He, Dror Wertheimer, and Arun Kumar. 2024. Table-GPT: Table Fine-tuned GPT for Diverse Table Tasks. *arXiv preprint arXiv:2310.09263* (2024).

[18] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36. 46534–46594.

[19] Michał Pietruszka, Maciej Półtorak, and Tomasz Rutkowski. 2025. Can Large Language Models Perform Feature Engineering? An Empirical Evaluation on Tabular Benchmarks. *arXiv preprint arXiv:2501.08240* (2025).

[20] Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. 2025. TabICL: A Tabular Foundation Model for In-Context Learning on Large Data. *arXiv preprint arXiv:2502.05564* (2025).

[21] Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. 2026. TabICLv2: A better, faster, scalable, and open tabular foundation model. *arXiv preprint arXiv:2602.11139* (2026).

[22] Michael Schmidt and Hod Lipson. 2009. Distilling free-form natural laws from experimental data. *Science* 324, 5923 (2009), 81–85.

[23] Parshin Shojaee, Kazem Meidani, Amir Barati Farimani, and Chandan K. Reddy. 2025. LLM-SR: Scientific Equation Discovery via Programming with Large Language Models. *arXiv preprint arXiv:2404.18400* (2025).

[24] Parshin Shojaee, Kazem Meidani, and Chandan K. Reddy. 2025. LLM-SRBench: A New Benchmark for Scientific Equation Discovery with Large Language Models. *arXiv preprint arXiv:2502.04415* (2025).

[25] Theresa Stadler, Bristena Oprisanu, and Carmela Troncoso. 2022. Synthetic data–anonymisation groundhog day. *31st USENIX Security Symposium (USENIX Security 22)* (2022), 1451–1468.

[26] Yuan Sui, Mengyu Zhou, Mingjie Zhou, Shi Han, and Dongmei Zhang. 2024. Table Meets LLM: Can Large Language Models Understand Structured Table Data? A Benchmark and Empirical Study. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining (WSDM)*. 645–654.

[27] Silviu-Marian Udrescu and Max Tegmark. 2020. AI Feynman: A physics-inspired method for symbolic regression. *Science Advances* 6, 16 (2020), eaay2631.

[28] Joaquin Vanschoren, Jan N. van Rijn, Bernd Bischl, and Luis Torgo. 2014. OpenML: Networked science in machine learning. *ACM SIGKDD Explorations Newsletter* 15, 2 (2014), 49–60.

[29] Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. 2019. Modeling tabular data using conditional GAN. *Advances in Neural Information Processing Systems (NeurIPS)* 32 (2019), 7335–7345.

[30] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.